

Session : Principale
Régime : RM
Enseignant : Sofiane HACHICHA
Classe(s) : CPI2
Barème : 4 + 16

Date : 17/03/2023
Matière : Programmation Java 2
Durée : 1h
Documents : Non autorisés
Nombre des pages : 5

Devoir Surveillé

Exercice 1

Qu'affiche le programme suivant ?

```
import java.util.*;
public class TestDevoir {
    public static void main(String[] args) {
        List<String> l0 = Arrays.asList("maman", "bateau", "ananas");
        List<String> l1 = List.copyOf(l0);
        try {
            l0.set(0, "aabam");
            System.out.println("Affichage 1");
            l0.forEach(System.out::println);
            System.out.println("Affichage 2");
            l1.forEach(System.out::println);
            l1.add("avion");
            System.out.println("Affichage 3");
            l1.forEach(System.out::println);
        } catch (UnsupportedOperationException e) {
            System.err.println("Opération non supportée");
        }
        l1.stream()
            .map(e -> e.substring(1, 3))
            .filter(c -> c.startsWith("a"))
            .forEach(System.out::println);
    }
}
```

Exercice 2

On souhaite modéliser un registre de patients qui sont les clients d'une pharmacie.

Chaque patient est représenté par son nom, son code et son âge. Deux patients sont égaux s'ils ont le même nom, le même code et le même âge.

Un médicament est représenté par son nom, sa quantité et son prix unitaire. On suppose que les médicaments sont comparés uniquement selon leurs prix unitaires.

Un registre pharmaceutique est caractérisé par un registre de type Map (la partie clé est Patient, la partie valeur est un ArrayList de Medicament) qui permet de créer initialement une Map vide tout en conservant l'ordre d'insertion des éléments qui seront ajoutés par la suite.

Une partie du code des classes **Patient**, **Medicament** et **RegistrePharmacie** vous est fournie. A vous de compléter le corps des méthodes signalées.

1. Compléter la classe **Patient** donnée ci-dessous.

```
import java.util.*;
class Patient {
    private String nom;
    private int code;
    private int age;

    public Patient(String nom, int code, int age) {
        this.nom = nom;
        this.code = code;
        this.age = age;
    }
    public String getNom() {
        return nom;
    }
    public void setNom(String nom) {
        this.nom = nom;
    }
    public int getCode() {
        return code;
    }
    public void setCode(int code) {
        this.code = code;
    }
    public int getAge() {
        return age;
    }
    public void setAge(int age) {
        this.age = age;
    }
}

@Override
public int hashCode() {
    // A compléter
}

@Override
public boolean equals(Object obj) {
    // A compléter
}

public String toString(){
    return "Patient : "+getNom()+" : "+getCode()+" : "+getAge();
}
}
```

2. Compléter la classe **Medicament** donnée ci-dessous.

```
import java.util.*;
class Medicament implements Comparable<Medicament> {
    private String nomMed;
    private int quantite;
    private double prix;

    public Medicament(String nomMed, int quantite, double prix) {
        this.nomMed = nomMed;
        this.quantite = quantite;
    }
}
```

```

        this.prix = prix;
    }
    public String getNomMed() {
        return nomMed;
    }
    public void setNomMed(String nomMed) {
        this.nomMed = nomMed;
    }
    public int getQuantite() {
        return quantite;
    }
    public void setQuantite(int quantite) {
        this.quantite = quantite;
    }
    public double getPrix() {
        return prix;
    }
    public void setPrix(double prix) {
        this.prix = prix;
    }
}

/* Retourne un ArrayList d'un médicament créé selon le constructeur de cette
classe */
    public ArrayList<Medicament> getListMedicament() {
        // A compléter
    }

/* Retourne un tableau de chaînes de caractères (tab[0]=nomMed, tab[1]=quantite et
tab[2]=prix) */
    public String[] getTabMedicament() {
        // A compléter
    }

    @Override
    public int hashCode() {
        // A compléter
    }

    @Override
    public boolean equals(Object obj) {
        // A compléter
    }

    @Override
    public int compareTo(Medicament o) {
        // A compléter
    }

    public String toString() {
        return "Médicament : "+getNomMed()+" : "+getQuantite()+" : "+getPrix();
    }
}

```

3. Compléter la classe **RegistrePharmacie** signalée ci-dessous.

```

import java.util.*;
class RegistrePharmacie {

    Map<Patient, ArrayList<Medicament>> registre;

```

```
public RegistrePharmacie() {
    // A compléter
}

/* Ajoute dans le registre un patient p ayant une ordonnance d'un médicament ord
sous forme d'un tableau de chaînes de caractères (ord[0]=nomMed,
ord[1]=quantite et ord[2]=prix) */
public void ajoutPatientMedicament(Patient p, String[] ord) {
    // A compléter
}

/* Affiche les noms des patients du registre */
public void afficheNomPatient() {
    // A compléter
}

/* Retourne une liste des noms des médicaments pris par le (les) patient(s) ayant
le nom nomPatient donné */
public List<String> getNomMedPatient(String nomPatient) {
    // A compléter
}

/* Retourne une liste triée contenant les codes des patients prenant le médicament
nomMedicament */
public List<Integer> getCodePatientMed(String nomMedicament){
    // A compléter
}

/* Retourne le prix total des médicaments d'un patient p */
public double getPrixMedicament(Patient p){
    // A compléter
}

/* Retourne une liste des noms des patients ayant le prix total des médicaments le
plus élevé */
public List<String> getNomPatientMed(){
    // A compléter
}

/* Retourne le nombre de patients qui ont un âge supérieur ou égal au paramètre
seuil donné. Pour un seuil non entier, cette méthode doit retourner -1 */
public int getCountPatientAge(String seuil){
    // A compléter
}
}
```

BON TRAVAIL

Indications :

L'interface `Collection<E>` définit plusieurs méthodes pour gérer des collections de type (`List<E>`, `Set<E>` et `Queue<E>`). Voici quelques méthodes fournies par cette interface :

Méthode	Rôle
<code>boolean add(E e)</code>	Ajouter un élément <code>e</code> à la collection
<code>boolean contains(Object o)</code>	Retourner un booléen qui précise si l'objet <code>o</code> est présent dans la collection
<code>boolean isEmpty()</code>	Retourner un booléen qui précise si la collection est vide
<code>Iterator<E> iterator()</code>	Retourner un <code>Iterator</code> qui parcourt les éléments de la collection
<code>int size()</code>	Retourner le nombre d'éléments contenus dans la collection
<code>Object[] toArray()</code>	Retourner un tableau contenant tous les éléments de la collection

L'interface `Map<K, V>` définit plusieurs méthodes telles que :

Méthode	Rôle
<code>boolean containsKey(Object key)</code>	Indiquer si la clé est contenue dans la Map
<code>boolean containsValue(Object value)</code>	Indiquer si la valeur est contenue dans la Map
<code>Set<Map.Entry<K,V>> entrySet()</code>	Renvoyer un ensemble contenant les paires clé/valeur de la Map
<code>Set<K> keySet()</code>	Renvoyer un ensemble contenant les clés de la Map
<code>V put(K key, V value)</code>	Insérer la clé et sa valeur associée passées en paramètres
<code>V get(K key)</code>	Renvoyer la valeur associée à la clé passée en paramètre
<code>Collection<V> values()</code>	Renvoyer une collection contenant les valeurs des éléments
<code>int size()</code>	Renvoyer le nombre d'éléments de la Map

L'interface `Map<K,V>` définit aussi une interface interne `Map.Entry<K,V>` qui fournit les fonctionnalités pour un objet encapsulant une paire clé/valeur et qui propose les méthodes `K getKey()` et `V getValue()` renvoyant respectivement la clé et la valeur correspondantes à cette `Map.Entry`.